Taller Practico: Arquitectura de Componentes en Java (.jar)

Informe de resultados — Microclase: Componentes

Integrantes del grupo

Kevin Sneider Zambrano Matiz

Brayan David Santos Alvarez

Sergio Steven Vanegas Cuervo

Contenido: (a) error inicial de empaquetado de peto y botas, (b) ejecucion inicial y analisis de fallas, (c) refactorizacion, librerias .jar cargadas y salida final, (d) tabla de metricas con procedimiento.

Parte 1. Diagnostico inicial

1.1 Empaquetado de Peto y Botas

(a) Error al intentar compilar/exportar cada modulo por separado, sin enlazar el otro:

```
$ javac -d out modulo-peto/src/peto/PetoEnlace.java   (sin modulo-botas)
inicial/modulo-peto/src/peto/PetoEnlace.java:3: error: package botas does not exist
import botas.BotasEnlace;
            ^
inicial/modulo-peto/src/peto/PetoEnlace.java:6: error: cannot find symbol
    public boolean escudoActivo(BotasEnlace botas) {
                                ^
  symbol:   class BotasEnlace
  location: class PetoEnlace
2 errors

$ javac -d out modulo-botas/src/botas/BotasEnlace.java   (sin modulo-peto)
inicial/modulo-botas/src/botas/BotasEnlace.java:3: error: package peto does not exist
import peto.PetoEnlace;
           ^
inicial/modulo-botas/src/botas/BotasEnlace.java:10: error: cannot find symbol
    public boolean sincronizar(PetoEnlace peto) {
                               ^
  symbol:   class PetoEnlace
  location: class BotasEnlace
2 errors

$ javac -d out modulo-peto/... modulo-botas/...   (juntos: compila, pero ya no son independientes)
OK solo si se compilan juntos
```

Por que ocurre. *PetoEnlace* importa *botas.BotasEnlace* y *BotasEnlace* importa *peto.PetoEnlace*: cada clase necesita a la otra para compilar. Al aislar un modulo, el compilador no encuentra el paquete del otro (*package botas does not exist* / *package peto does not exist*). Solo compilan si se ponen juntos, lo que significa que en realidad son un unico componente disfrazado de dos.

Por que impide generar los .jar de forma independiente. Un .jar se construye a partir de clases ya compiladas. Como ninguno de los dos modulos compila sin el otro, no existe un orden valido de construccion (huevo y gallina): ni peto.jar ni botas.jar pueden generarse, versionarse o reutilizarse por separado.

Principio incumplido: ADP (Acyclic Dependencies Principle). El grafo de dependencias entre componentes debe ser aciclico; aqui peto → botas → peto forma un ciclo. Esto produce el **sindrome de la manana siguiente**: lo que funcionaba ayer deja de compilar hoy porque alguien modifico el otro componente del ciclo. Como cada cambio en uno puede romper al otro, los dos equipos quedan obligados a integrarse, probarse y liberarse en bloque.

1.2 Ejecucion de los demas modulos (espada y casco)

(b) Salida inicial de *Main.java* con espada.jar y casco.jar importados en juego-principal:

```
$ java -cp espada.jar:casco.jar:out app.Main
=== PRUEBA INICIAL: COMPONENT QUEST ===
Ataque con espada: -50 pts
Defensa del casco: CASCO DESTRUIDO: No soporta ataques de FUEGO
```

Que sucede. El programa compila y corre, pero el comportamiento es incorrecto: la espada ataca con **-50 pts** (50 de dano menos 100 del peso de la armadura que lleva pegada) y el casco es destruido por el ataque de FUEGO. El defecto no es un error de sintaxis sino de diseno de componentes.

modulo-espada: CRP, CCP y REP.

- **CRP (Common Reuse):** quien reutiliza *EspadaExplorador* queda forzado a arrastrar *ArmaduraMarmol*, una clase que no necesita (y que ademas le resta dano).
- **CCP (Common Closure):** espada y armadura cambian por razones distintas, por lo que no deberian estar agrupadas en el mismo componente.
- **REP (Reuse/Release Equivalence):** el componente no es una unidad coherente de reutilizacion y liberacion: un release de «espada» incluye una armadura sin relacion.

modulo-casco: SAP y SDP.

- **SAP (Stable Abstractions):** un componente debe ser tan abstracto como estable. *CascoHierroSellado* es muy estable (mucho depende de el, nada de el) pero totalmente concreto, y en el mismo componente vive *GemaOlvidada*, una abstraccion que nadie usa.
- **SDP (Stable Dependencies):** al ser estable y concreto, el casco no puede extenderse sin modificarlo; cualquier cambio (un nuevo elemento) se propaga a todos los que dependen de el, y la estabilidad que tiene no esta respaldada por abstracciones que permitan cambiar sin romperlo.

Ubicacion en la grafica Abstraccion vs. Inestabilidad. *CascoHierroSellado* (I = 0, A = 0) esta en la **Zona de Dolor**: estable y concreto, rigido. *GemaOlvidada* (aislada, A = 1, I = 1 por convencion) esta en la **Zona de Inutilidad**: abstracta pero sin ningun dependiente que la aproveche.

Parte 2. Refactorizacion y nuevos componentes

Ronda	Cambio realizado	Componente
1 – Espada	Se elimina *ArmaduraMarmol*. *EspadaExplorador.atacar()* devuelve 50 sin dependencias extra.	espada-v1.0.jar
2 – Peto y Botas	Nuevo *modulo-core* con la interfaz *core.GemaEnergia { boolean tieneCarga(); }*. *BotasEnlace* la implementa; *PetoEnlace* la recibe por constructor (o *conectarFuente*). Se eliminan los import cruzados. El escudo base del peto siempre esta activo; con fuente cargada ademas se refuerza, y con fuente *null* sigue activo.	core-v1.0.jar peto-v1.0.jar botas-v1.0.jar
4 – Casco	Se elimina *GemaOlvidada*. Nueva interfaz *ProteccionElemental*; la implementan *CascoHierroSellado* (fisico) y la nueva *CascoRunico* (magico/elemental, p. ej. FUEGO).	casco-v1.0.jar

Dependencias resultantes entre componentes (aciclicas):

```
juego-principal --> espada-v1.0
juego-principal --> peto-v1.0  --> core-v1.0
juego-principal --> botas-v1.0 --> core-v1.0
juego-principal --> casco-v1.0
(peto-v1.0 y botas-v1.0 ya no se conocen entre si)
```

3. Integracion en juego-principal

(c) Librerias .jar cargadas en juego-principal (solo estos 5 archivos, sin copiar clases fuente):

```
juego-principal/libs/
  core-v1.0.jar    -> core/GemaEnergia
  espada-v1.0.jar  -> armas/EspadaExplorador
  peto-v1.0.jar    -> peto/PetoEnlace
  botas-v1.0.jar   -> botas/BotasEnlace
  casco-v1.0.jar   -> casco/ProteccionElemental, CascoHierroSellado, CascoRunico
```

Salida exitosa de *Main.java* actualizado, con los tres escenarios:

```
$ java -cp core-v1.0.jar:espada-v1.0.jar:peto-v1.0.jar:botas-v1.0.jar:casco-v1.0.jar:out app.Main
=== PRUEBA FINAL: COMPONENT QUEST (componentes .jar v1.0) ===

--- Ronda 1: Dragon Ignis ---
Ataque con espada: 50 pts

--- Ronda 2: Golem Sismico ---
Con botas -> escudo activo: true | escudo reforzado: true
Sin botas -> escudo activo: true | escudo reforzado: false

--- Ronda 4: Espectro Cambiante (FUEGO) ---
CascoHierroSellado vs FISICO: Golpe fisico bloqueado
CascoHierroSellado vs FUEGO : CASCO DESTRUIDO: No soporta ataques de FUEGO
CascoRunico        vs FUEGO : Ataque de FUEGO resistido por las runas
```

Lectura de resultados: la espada ataca limpio (50 pts); el peto mantiene el escudo activo aun al retirar las botas en combate (solo pierde el refuerzo); y el casco runico resiste el FUEGO que destruia al casco de hierro, manteniendo el casco de hierro para golpes fisicos.

Parte 3. Metricas de componentes

Formulas: **I = Ce / (Ca + Ce)**, **A = Na / Nc**, **D = |A + I – 1|**. Ca = dependencias entrantes, Ce = dependencias salientes, Na = interfaces/clases abstractas, Nc = total de clases del componente (la interfaz cuenta como una de ellas).

Componente	Ca	Ce	I	A	D	Zona / Diagnostico
core-v1.0.jar (Final)	2	0	0	1	0	Secuencia Principal
peto-v1.0.jar (Final)	1	1	0.5	0	0.50	Bajo la Secuencia Principal (hacia Zona de Dolor, riesgo moderado)
casco-v1.0.jar (Final)	1	0	0	0.33	0.67	Zona de Dolor
CascoHierroSellado (Inicial)	3	0	0	0	1	Zona de Dolor
GemaOlvidada (Inicial)	0	0	1	1	1	Zona de Inutilidad

Procedimiento de cada calculo

core-v1.0.jar (Final)

Ca = 2 (peto, botas), Ce = 0. Clases: GemaEnergia (interfaz) → Na = 1, Nc = 1.
I = 0 / (2 + 0) = **0** | A = 1 / 1 = **1** | D = |1 + 0 – 1| = **0**.
Diagnostico: **sobre la Secuencia Principal**. Es maximamente estable y totalmente abstracto: el lugar ideal para lo que muchos usan.

peto-v1.0.jar (Final)

Ca = 1 (app), Ce = 1 (core). Clases: PetoEnlace (concreta) → Na = 0, Nc = 1.
I = 1 / (1 + 1) = **0.5** | A = 0 / 1 = **0** | D = |0 + 0.5 – 1| = **0.5**.
Diagnostico: **por debajo de la Secuencia Principal** (A + I < 1): componente concreto y de estabilidad intermedia. No esta en la esquina de la Zona de Dolor (I = 0, A = 0) y su unica dependencia es hacia una abstraccion, pero si crece o gana mas dependientes se desplazaria hacia esa zona.

casco-v1.0.jar (Final)

Ca = 1 (app), Ce = 0. Clases: ProteccionElemental (interfaz), CascoHierroSellado y CascoRunico → Na = 1, Nc = 3.
I = 0 / (1 + 0) = **0** | A = 1 / 3 = **0.33** | D = |0.33 + 0 – 1| = **0.67**.
Diagnostico: **Zona de Dolor** (aunque menos severa que antes): es estable pero mayormente concreto. Mejoro frente a la version inicial (D paso de 1 a 0.67) gracias a la interfaz, que permite agregar cascos nuevos sin modificar los existentes.

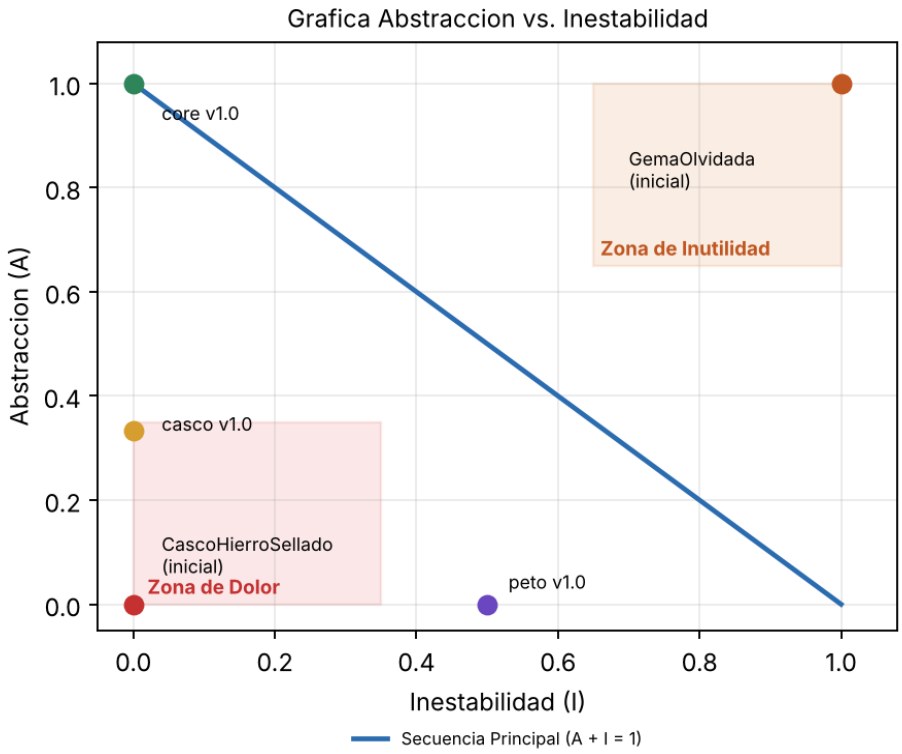
CascoHierroSellado (Inicial, Ca = 3)

Ca = 3, Ce = 0. Clase concreta → Na = 0, Nc = 1.
I = 0 / (3 + 0) = **0** | A = 0 / 1 = **0** | D = |0 + 0 – 1| = **1**.
Diagnostico: **Zona de Dolor** en su punto extremo: estable, concreto y rigido; cambiarlo afecta a los tres que dependen de el.

GemaOlvidada (Inicial, aislada)

Ca = 0, Ce = 0. Interfaz → A = Na / Nc = 1 / 1 = **1**.
I = 0 / (0 + 0) queda indeterminada (0/0). Por convencion se toma **I = 1**: nada depende de ella, asi que no aporta estabilidad.
D = |1 + 1 – 1| = **1**.
Diagnostico: **Zona de Inutilidad**: abstraccion sin ningun uso.

Grafica Abstraccion vs. Inestabilidad



Nota: la ubicacion de GemaOlvidada usa la convencion I = 1 para componentes aislados; con I = 0 su distancia seria 0 y quedaria sobre la Secuencia Principal, lo cual no refleja que nada la utiliza.